Módulo 53 | Pronóstico de ventas e intervalos de confianza

Javier Bueno Sánchez · Profesión Científico de Datos

Datos: Advertising.csv adjunto (200 observaciones). Reproducir el notebook colocando el CSV en la misma carpeta que el notebook. Inversión y ventas se expresan en las unidades originales del conjunto; no se especifica moneda.

Diseño. Separación aleatoria 70/30 con random_state=1. Selección hacia atrás por p-valor bilateral, α=0.10, exclusivamente sobre entrenamiento; intercepto obligatorio. Contrasto coeficientes, rendimiento fuera de muestra, incertidumbre y supuestos. Las pruebas son diagnósticos, no garantías de causalidad ni de rendimiento futuro.

Código ejecutado

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from pathlib import Path
from scipy import stats
import statsmodels.api as sm
from statsmodels.stats.diagnostic import het_white, het_breuschpagan, linear_reset
from statsmodels.stats.stattools import durbin_watson, jarque_bera, omni_normtest
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
pd.set_option('display.float_format',lambda v:f'{v:,.5f}')
df=pd.read_csv('Advertising.csv'); print('Forma:',df.shape,'; duplicados:',df.duplicated().sum())
print('Faltantes:',df.isna().sum().to_dict()); display(df.describe().T)
assert df.shape==(200,4) and not df.isna().any().any()
Xtr,Xte,ytr,yte=train_test_split(df[['TV','Radio','Newspaper']],df.Sales,test_size=.30,random_state=1)
print('Entrenamiento:',len(Xtr),'Prueba:',len(Xte))
```

Salida

```
Forma: (200, 4) ; duplicados: 0
Faltantes: {'TV': 0, 'Radio': 0, 'Newspaper': 0, 'Sales': 0}
              count      mean      std     min      25%       50%       75%       max
TV        200.00000 147.04250 85.85424 0.70000 74.37500 149.75000 218.82500 296.40000
Radio     200.00000  23.26400 14.84681 0.00000  9.97500  22.90000  36.52500  49.60000
Newspaper 200.00000  30.55400 21.77862 0.30000 12.75000  25.75000  45.10000 114.00000
Sales     200.00000  15.13050  5.28389 1.60000 11.00000  16.00000  19.05000  27.00000
Entrenamiento: 140 Prueba: 60
```

1. Selección y ajuste

En cada iteración retiro el predictor con mayor p-valor superior a 0.10 y vuelvo a estimar. El resultado y los p-valores dependen de esta partición. El p-valor no mide el tamaño ni la utilidad empresarial del efecto.

Código ejecutado

```
selected=list(Xtr.columns); trace=[]
while True:
    fit=sm.OLS(ytr,sm.add_constant(Xtr[selected],has_constant='add')).fit()
    pv=fit.pvalues.drop('const'); worst=pv.idxmax()
    trace.append({'variables':', '.join(selected),'peor_variable':worst,'p':pv.max()})
    if pv.max()<=.10:break
    selected.remove(worst)
display(pd.DataFrame(trace)); print('Variables finales:',selected)
print(fit.summary())
predtr=fit.predict(sm.add_constant(Xtr[selected],has_constant='add'))
predte=fit.predict(sm.add_constant(Xte[selected],has_constant='add'))
for name,y,p in [('Entrenamiento',ytr,predtr),('Prueba',yte,predte)]:
    print(f'{name}: R²={r2_score(y,p):.4f}; MAE={mean_absolute_error(y,p):.4f};
RMSE={np.sqrt(mean_squared_error(y,p)):.4f}')
print('Ecuación: Sales = '+ ' + '.join(f'({fit.params[k]:.6f})·{k}' for k in fit.params.index))
```

Salida

```
              variables peor_variable       p
0  TV, Radio, Newspaper     Newspaper 0.83906
1             TV, Radio         Radio 0.00000
Variables finales: ['TV', 'Radio']
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.899
Model:                            OLS   Adj. R-squared:                  0.898
Method:                 Least Squares   F-statistic:                     612.0
Date:                Mon, 28 Sep 2026   Prob (F-statistic):           4.95e-69
Time:                        11:09:57   Log-Likelihood:                -272.37
No. Observations:                 140   AIC:                             550.7
Df Residuals:                     137   BIC:                             559.6
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.6390      0.350     13.240      0.000       3.946       5.332
TV             0.0550      0.002     32.903      0.000       0.052       0.058
Radio          0.1016      0.010     10.677      0.000       0.083       0.120
==============================================================================
Omnibus:                       14.841   Durbin-Watson:                   2.038
Prob(Omnibus):                  0.001   Jarque-Bera (JB):               24.663
Skew:                          -0.512   Prob(JB):                     4.41e-06
Kurtosis:                       4.783   Cond. No.                         407.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Entrenamiento: R²=0.8993; MAE=1.2709; RMSE=1.6932
Prueba: R²=0.9074; MAE=1.1920; RMSE=1.5377
Ecuación: Sales = (4.639009)·const + (0.055005)·TV + (0.101644)·Radio
```

2. Pronóstico para TV=100, Radio=50 y Newspaper=70

Si Newspaper se excluye, su valor 70 no entra en la ecuación: el pronóstico es el mismo para cualquier inversión en periódico manteniendo TV y Radio fijos. El intervalo de confianza estima la media condicional; añado el intervalo de predicción para una observación nueva. Ambos son bilaterales al 90%. Un predictor puede estar fuera de su rango marginal, y la combinación puede estar fuera del soporte conjunto observado.

Código ejecutado

```
scenario={'TV':100.,'Radio':50.,'Newspaper':70.}
x0=sm.add_constant(pd.DataFrame([{k:scenario[k] for k in selected}]),has_constant='add')
pr=fit.get_prediction(x0).summary_frame(alpha=.10)
display(pr[['mean','mean_ci_lower','mean_ci_upper','obs_ci_lower','obs_ci_upper']])
v=x0.to_numpy()[0]; inv=np.linalg.inv(fit.model.exog.T@fit.model.exog)
yhat=float(v@fit.params.to_numpy()); se_mean=np.sqrt(fit.mse_resid*(v@inv@v))
tcrit=stats.t.ppf(.95,fit.df_resid)
print('Verificación matricial IC media:',(yhat-tcrit*se_mean,yhat+tcrit*se_mean))
print('Verificación matricial IP individual:',(yhat-
tcrit*np.sqrt(fit.mse_resid+se_mean**2),yhat+tcrit*np.sqrt(fit.mse_resid+se_mean**2)))
for c in selected:print(c,'rango entrenamiento:',Xtr[c].min(),Xtr[c].max(),'escenario:',scenario[c])
center=Xtr[selected].mean().to_numpy(); cov=np.cov(Xtr[selected].to_numpy(),rowvar=False)
dist=(np.array([scenario[c] for c in
selected])-center)@np.linalg.inv(np.atleast_2d(cov))@(np.array([scenario[c] for c in selected])-center)
print('Distancia Mahalanobis² del escenario:',dist,'; percentil 95 observado:',np.percentile([((row-
```

```
center)@np.linalg.inv(np.atleast_2d(cov))@(row-center)) for row in Xtr[selected].to_numpy()],95))
```

Salida

```
      mean  mean_ci_lower  mean_ci_upper  obs_ci_lower  obs_ci_upper
0 15.22168       14.71820       15.72515      12.34281      18.10054
Verificación matricial IC media: (np.float64(14.718204466533187), np.float64(15.72514686322122))
Verificación matricial IP individual: (np.float64(12.342813225953813), np.float64(18.100538103800595))
TV rango entrenamiento: 0.7 296.4 escenario: 100.0
Radio rango entrenamiento: 0.0 49.4 escenario: 50.0
Distancia Mahalanobis² del escenario: 3.3925757276474147 ; percentil 95 observado: 4.0486852209327715
```

3. Supuestos y pruebas (α=0.10)

Los diagnósticos se calculan sobre residuos de entrenamiento del modelo final. Normalidad: Jarque–Bera, Shapiro y gráfico Q–Q; homocedasticidad: White; autocorrelación: Durbin–Watson y Breusch–Godfrey; multicolinealidad: VIF; especificación de la media: RESET. El orden de filas del CSV no representa tiempo: DW/BG son diagnósticos descriptivos condicionados al orden, y no constituyen una prueba válida de independencia temporal sin secuencia temporal real. También se inspecciona influencia y forma funcional.

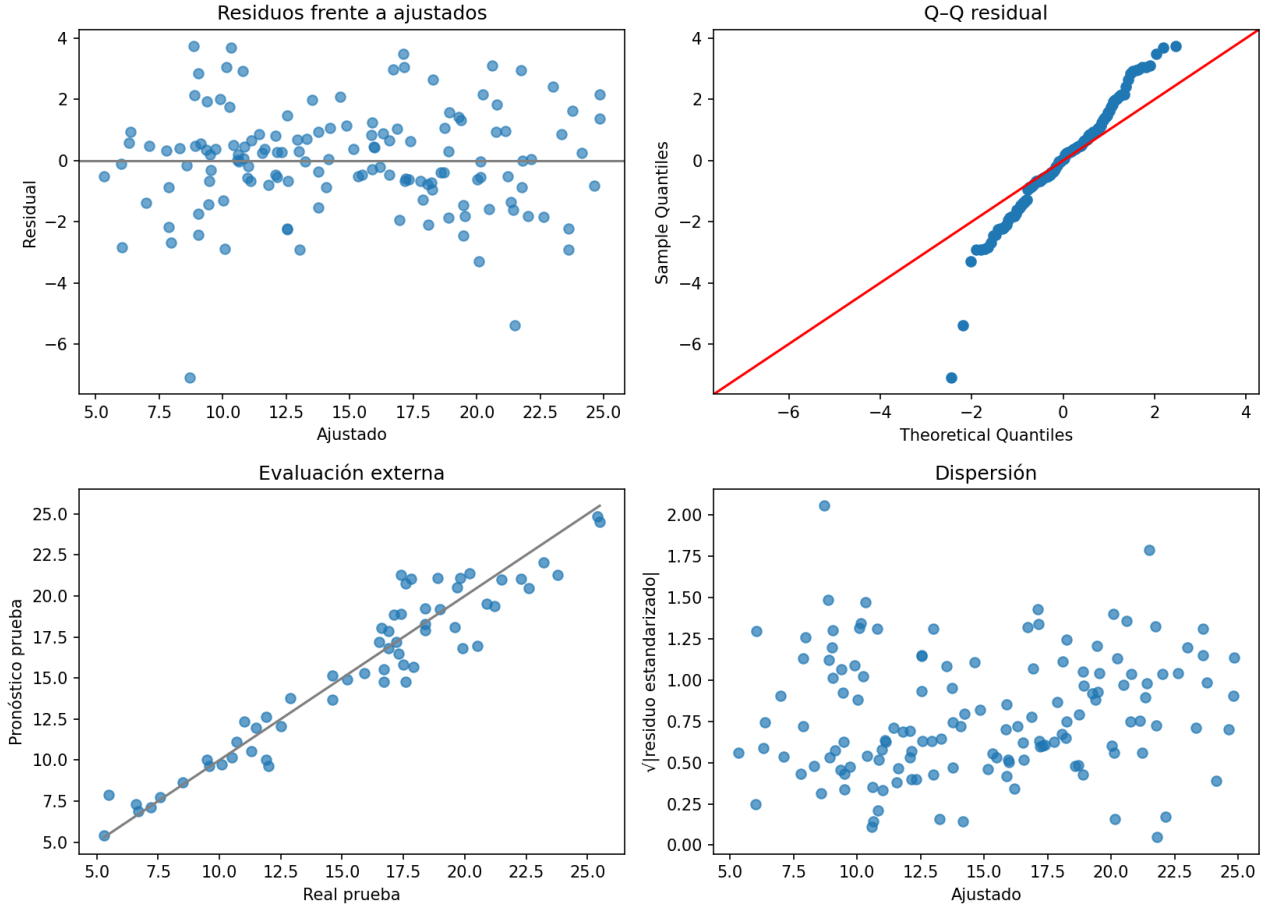
Código ejecutado

```
from statsmodels.stats.diagnostic import acorr_breusch_godfrey
e=fit.resid.to_numpy(); ex=fit.model.exog; nobs=len(e)
jb=jarque_bera(e); sh=stats.shapiro(e); wh=het_white(e,ex); bp=het_breuschpagan(e,ex)
bg=acorr_breusch_godfrey(fit,nlags=1); reset=linear_reset(fit,power=2,use_f=True)
vif=pd.Series({selected[j-1]:variance_inflation_factor(ex,j) for j in range(1,ex.shape[1])})
tests=pd.DataFrame([('Jarque–Bera normalidad',jb[0],jb[1]),('Shapiro–Wilk
normalidad',sh.statistic,sh.pvalue),('White heterocedasticidad LM',wh[0],wh[1]),('Breusch–Pagan
heterocedasticidad LM',bp[0],bp[1]),('Breusch–Godfrey orden 1',bg[0],bg[1]),('RESET
especificación',float(reset.fvalue),float(reset.pvalue))],columns=['Prueba','Estadístico','p-valor'])
tests['Interpretación α=.10']=np.where(tests['p-valor']<.10,'Rechazo H■','No rechazo H■')
display(tests); print('Durbin–Watson (orden original conservado en
train):',durbin_watson(e));print('VIF:',vif.to_dict());print('Sesgo
scipy:',stats.skew(e,bias=True),'exceso curtosis scipy:',stats.kurtosis(e,fisher=True,bias=True))
infl=fit.get_influence(); cooks=infl.cooks_distance[0]
print('Cook D máximo:',max(cooks),'casos > 4/n:',sum(cooks>4/nobs))
fig,axs=plt.subplots(2,2,figsize=(11,8));axs[0,0].scatter(fit.fittedvalues,e,alpha=.65);axs[0,0].axhline(
0,color='gray');axs[0,0].set(xlabel='Ajustado',ylabel='Residual',title='Residuos frente a ajustados')
sm.qqplot(e,line='45',ax=axs[0,1]);axs[0,1].set_title('Q–Q residual')
axs[1,0].scatter(yte,predte,alpha=.7);axs[1,0].plot([min(yte),max(yte)],[min(yte),max(yte)],color='gray')
;axs[1,0].set(xlabel='Real prueba',ylabel='Pronóstico prueba',title='Evaluación externa')
axs[1,1].scatter(fit.fittedvalues,np.sqrt(np.abs(infl.resid_studentized_internal)),alpha=.7);axs[1,1].set
(xlabel='Ajustado',ylabel='√|residuo estandarizado|',title='Dispersión')
fig.tight_layout();fig.savefig('diagnosticos_M53.png',dpi=150);plt.show()
```

Salida

```
<string>:4: FutureWarning: acorr_breusch_godfrey currently returns a plain tuple whose length depends
on the store argument. In release 0.16 or after July 2027, whichever is later, the default behavior
will switch to always returning an LMTestResult NamedTuple. Set result_object=True to switch now, or
result_object=False to keep the current behavior and silence this warning.
                              Prueba  Estadístico  p-valor Interpretación α=.10
0               Jarque–Bera normalidad     24.66286  0.00000           Rechazo H■
1              Shapiro–Wilk normalidad      0.96955  0.00323           Rechazo H■
2          White heterocedasticidad LM     11.35338  0.04481           Rechazo H■
3  Breusch–Pagan heterocedasticidad LM      0.00453  0.99774        No rechazo H■
4              Breusch–Godfrey orden 1      0.07318  0.78676        No rechazo H■
5                 RESET especificación      1.12154  0.29146        No rechazo H■
Durbin–Watson (orden original conservado en train): 2.0375161916374998
VIF: {'TV': 1.0013523545176488, 'Radio': 1.0013523545176493}
```

```
Sesgo scipy: -0.5124182186793587 exceso curtosis scipy: 1.7825906401266707
Cook D máximo: 0.21989405383193023 casos > 4/n: 9
```



4. Fórmulas implementadas y comparación

Convenciones: sesgo $g_1=m_3/m_2^{3/2}$ y **exceso** de curtosis $g_2=m_4/m_2^2-3$, con $m_k=n^{-1}\sum(e_i-\bar e)^k$ (equivalen a SciPy `bias=True`). DW=$\sum_{i=2}^n(e_i-e_{i-1})^2/\sum_{i=1}^n e_i^2$. White LM=$nR^2$ de una regresión auxiliar de e_i^2 contra intercepto, predictores, sus cuadrados y todas las interacciones; rango y grados de libertad efectivos se determinan por el diseño auxiliar. Como el entrenamiento conserva el orden que devuelve `train_test_split`, DW usa ese orden arbitrario.

Código ejecutado

```
from itertools import combinations_with_replacement

def sesgo_manual(a):

    z=np.asarray(a)-np.mean(a); m2=np.mean(z**2);return np.mean(z**3)/m2**1.5

def curtosis_exceso_manual(a):

    z=np.asarray(a)-np.mean(a);m2=np.mean(z**2);return np.mean(z**4)/m2**2-3

def dw_manual(a):

    a=np.asarray(a);return np.sum(np.diff(a)**2)/np.sum(a*a)

def white_manual(a,design):

    design=np.asarray(design);raw=design[:,1:]

    aux=np.column_stack([np.ones(len(a)),raw]+[raw[:,i]*raw[:,j] for i,j in

combinations_with_replacement(range(raw.shape[1]),2)])

    beta=np.linalg.lstsq(aux,np.asarray(a)**2,rcond=None)[0]

    fitted=aux@beta; target=np.asarray(a)**2

    r2=1-np.sum((target-fitted)**2)/np.sum((target-target.mean())**2)

    lm=len(a)*r2; dof=np.linalg.matrix_rank(aux)-1

    return lm,stats.chi2.sf(lm,dof),dof

manual={'Sesgo':sesgo_manual(e),'Exceso

curtosis':curtosis_exceso_manual(e),'Durbin–Watson':dw_manual(e),'White LM':white_manual(e,ex)[0],'White

p':white_manual(e,ex)[1]}

library={'Sesgo':stats.skew(e,bias=True),'Exceso
```

```
curtosis':stats.kurtosis(e,fisher=True,bias=True),'Durbin–Watson':durbin_watson(e),'White
LM':wh[0],'White p':wh[1]}
comparison=pd.DataFrame({'Manual':manual,'Librería':library});comparison['Diferencia
abs']=(comparison.Manual-comparison.Librería).abs()
display(comparison);print('White gl manual:',white_manual(e,ex)[2],'White gl
librería:',ex.shape[1]*(ex.shape[1]+1)//2-1)
assert np.allclose(comparison.Manual,comparison.Librería,rtol=1e-8,atol=1e-8)
```

Salida

```
                   Manual  Librería  Diferencia abs
Sesgo           -0.51242  -0.51242         0.00000
Exceso curtosis  1.78259   1.78259         0.00000
Durbin–Watson    2.03752   2.03752         0.00000
White LM        11.35338  11.35338         0.00000
White p          0.04481   0.04481         0.00000
White gl manual: 5 White gl librería: 5
```

5. Dictamen e intervalo robusto complementario

White rechaza homocedasticidad (p≈0.045), mientras Breusch–Pagan no la rechaza: detectan formas distintas de varianza condicional. Jarque–Bera y Shapiro rechazan normalidad residual. Por ello el intervalo clásico de Student es el resultado solicitado, pero su cobertura nominal del 90% no queda validada. Se presenta un intervalo HC3 para la media como análisis de sensibilidad asintótico; tampoco corrige por sí solo la selección de variables ni produce un intervalo individual calibrado. La muestra no tiene dimensión temporal identificada; DW≈2 no prueba ausencia de dependencia temporal. RESET no detecta mala especificación a α=.10, sin demostrar que la forma funcional sea correcta.

Código ejecutado

```
rob=fit.get_robustcov_results(cov_type='HC3')
se_hc3=np.sqrt(float(v@rob.cov_params()@v))
ci_hc3=(yhat-stats.norm.ppf(.95)*se_hc3,yhat+stats.norm.ppf(.95)*se_hc3)
print('HC3, IC asintótico 90% para media:',ci_hc3)
print('Conclusión: R² prueba',round(r2_score(yte,predte),4),'; IC clásico
90%',tuple(pr[['mean_ci_lower','mean_ci_upper']].iloc[0].round(4)), '; White p',round(wh[1],5))
```

Salida

```
HC3, IC asintótico 90% para media: (np.float64(14.646558172922056), np.float64(15.79679315683235))
Conclusión: R² prueba 0.9074 ; IC clásico 90% (14.7182, 15.7251) ; White p 0.04481
```

Interpretación integrada. La ecuación final utiliza TV y Radio; el efecto estimado del periódico no supera la selección al 10%. Los coeficientes representan asociaciones condicionales por una unidad adicional invertida, no retornos causales. La comparación R²/MAE/RMSE en prueba describe utilidad predictiva fuera del entrenamiento. La inversión de Radio=50 excede ligeramente el máximo observado en entrenamiento (49.4); corresponde señalar esta extrapolación marginal. Cook identifica observaciones para revisión, sin justificación para eliminarlas automáticamente. Fuentes metodológicas: documentación de statsmodels (OLS, get_prediction, het_white, durbin_watson) y SciPy (skew, kurtosis).